



INSTITUTO TECNOLÓGICO de Tuxtepec

“Curso Intensivo”

Presenta

Ferrer Pantoja Joshua Enrique

Carrera

ING. SISTEMAS COMPUTACIONALES

Docente:

O. Julio Aguilar Carmona

Tuxtepec, Oax.

05, Mayo 2026



Day 0 - Troubleshooting and FAQs

How do I add Google Cloud Services to my Kaggle Notebook via Kaggle Secrets?

You can watch the video timestamped here to your Kaggle Notebook as a Kaggle User Secret.

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Input

+ Add Input Upload

No input attached

Attach a Kaggle dataset, model, or competition

Output

/kaggle/working

Day 0 - Troubleshooting and FAQs

How do I add Google Cloud Services to my Kaggle Notebook via Kaggle Secrets?

You can watch the video timestamped here to your Kaggle Notebook as a Kaggle User Secret.

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Secrets

New secret

LABEL

GOOGLE_API_KEY

VALUE

secretpassword12345

Save Cancel

Day 0 - Troubleshooting and FAQs

```
After (with retry):

agent = LlmAgent(
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config # Adds automatic retry
    ),
    ...
)

# Example: Complete agent with retry configuration
from google.adk.agents import LlmAgent

agent = LlmAgent(
    name="my_agent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config, # Apply retry config here
    ),
    instruction="You are a helpful assistant.",
)

print("✅ Agent created with automatic retry on HTTP errors")
```

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

Add Secret Copy snippet

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Day 1a - From Prompt to Action

```
# See the LICENSE for the specific language governing permissions and
# limitations under the License.

pip install google-adk
```

Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable<=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.34.0)
Requirement already satisfied: google-cloud-discoveryengine<0.14.0,>=0.13.12 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.13.12)
Requirement already satisfied: google-cloud-secret-manager<3.0.0,>=2.22.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.25.0)
Requirement already satisfied: google-cloud-spanner<4.0.0,>=3.56.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (3.56.0)

Input

+ Add Input Upload

No input attached

Attach a Kaggle dataset, model, or competition

Output (76KiB / 19.5GiB)

/kaggle/working

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Google AI Studio

Claves de API

Guía de inicio rápido de la API

Crear clave de API

Agrupar por: API Key Proyecto

Filtrar por: Todos los proyectos

Clave	Proyecto	Creado	Nivel de facturación
...t2uU	Default Gemini Project	28 abr 2026	Configurar la facturación
Default Gemini API Key	gen-lang-client-0549976016		Nivel gratuito

¿No encuentras tus claves de API aquí?

Esta lista solo muestra las claves de API de los proyectos importados a Google AI Studio. Importa otros proyectos para administrar sus claves de API asociadas. También puedes crear una nueva clave de API arriba. [Más información](#)

Importar proyectos

UV alto
Ahora

Búsqueda

ESP LAA

05:25 p. m.
30/04/2026

Day 1a - From Prompt to Action

File Edit View Run Settings Add-ons Help

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print("Gemini API key setup complete.")
except Exception as e:
    print(f"Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets")

galerror Traceback (most recent call last)
/usr/lib/python3.11/urllib/request.py in do_open(self, http_class, req, **http_conn_args)
1347     try:
1348         h.request(req.get_method(), req.selector, req.data, headers,
1349                  encode_chunked=req.has_header('Transfer-encoding'))

/usr/lib/python3.11/http/client.py in request(self, method, url, body, headers, encode_chunked)
1302     """Send a complete request to the server."""
-> 1303     self._send_request(method, url, body, headers, encode_chunked)
1304

/usr/lib/python3.11/http/client.py in _send_request(self, method, url, body, headers, encode_chunked)
1348     body = _encode(body, 'body')
-> 1349     self.endheaders(body, encode_chunked=encode_chunked)
1350
```

Output (76KiB / 19.5GiB)

Table of contents

- Your First AI Agent: From Prompt to Action
- Please Read
- Get started with Kaggle Notebooks
- Section 1: Setup
- Section 2: Your first AI Agent with ADK

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

Learn more OK, Got it.

38°C
Soleado

Búsqueda

ESP LAA

05:30 p. m.
30/04/2026

Day 1a - From Prompt to Action

Draft saved

Share Save Version 0

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (14m)

Run All Code

3. Authenticate in the notebook

Run the cell below to complete authentication.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}")
    print(f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connect ion error trying to communicate with service.")
```

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

37°C Soleado

Búsqueda

ESP LAA

05:42 p. m. 30/04/2026

Notebook

Input

+ Add Input Upload

No input attached

Attach a Kaggle dataset, model, or competition

Output (76KiB / 19.5GiB)

/kaggle/working

Table of contents

Day 1a - From Prompt to Action

Draft saved

Share Save Version 0

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (16m)

Run All Code

4. Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import Agent
from google.adk.models.google_ilm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

+ Code + Markdown

1.4: Helper functions

We'll define some helper functions. If you are running this outside the Kaggle environment, you don't need to do this.

```
# Define helper functions that will be reused throughout the notebook

from IPython.core.display import display, HTML
from time import sleep
```

Add Secrets

Code Snippet

from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

37°C Soleado

Búsqueda

ESP LAA

05:44 p. m. 30/04/2026

Day 1a - From Prompt to Action Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (16m)

```

</ol>
<em style="font-size: 0.9em; color: #555;">(If you click the button before running the next
</div>
<a href='{url}' target='_blank' style="
display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
Open ADK Web UI (after running cell below) </a>
</div>
</a>

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

+ Code + Markdown

1.5: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

Add Secrets

Code Snippet

```

from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0

```

Copy to clipboard

GOOGLE_API_KEY *****

+ Add Secret + Copy snippet

37°C Soleado

Búsqueda

ESP LAA

05:44 p. m. 30/04/2026

```

root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")

```

✅ Root Agent defined.

+ Code + Markdown

2.3 Run your agent

Now it's time to bring your agent to life and send it a query. To do this, you need a `Runner`, which is the central component within ADK that acts as the orchestrator. It manages the conversation, sends our messages to the agent, and handles its responses.

a. Create an `InMemoryRunner` and tell it to use our `root_agent`:

```

runner = InMemoryRunner(agent=root_agent)

print("✅ Runner created.")

```

✅ Runner created.

+ Code + Markdown

👉 Note that we are using the Python Runner directly in this notebook. You can also run agents using ADK command-line tools such as

response = await runner.run_debug(
 "What is Agent Development Kit from Google? What languages is the SDK available in?"
)

Created new session: debug_session_id

User > What is Agent Development Kit from Google? What languages is the SDK available in?

response = await runner.run_debug("What's the weather in London?")

Continue session: debug_session_id

User > What's the weather in London?

!adk create sample-agent --model gemini-2.5-flash-lite --api_key \$GOOGLE_API_KEY

+ Code

+ Markdown

Get your custom URL to access the ADK web UI in the Kaggle Notebooks environment:

+ Code

+ Markdown

The screenshot shows a Kaggle Notebook interface. The main content area displays a 'Congratulations!' message for successfully building and running the first agent with ADK. It explains that the agent 'reasoned' and 'acted' based on a prompt. A 'Note: No submission required!' is also present. Below this, there is a 'Learn More' section with links to ADK Documentation, Quickstart for Python, Agents Overview, and Tools Overview. On the right side, a 'Secrets' panel is open, showing a 'Code Snippet' for importing and using secrets, and a list of secrets including 'GOOGLE_API_KEY' which is masked with asterisks. The bottom of the screen shows a Windows taskbar with the date 30/04/2026 and time 05:48 p.m.

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run the cell below to access the 'GOOGLE_API_KEY' you just saved and set it as an environment variable for the notebook to use:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}")
```

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

37°C Soleado

ESP LAA 05:52 p. m. 30/04/2026

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run the cell below to access the 'GOOGLE_API_KEY' you just saved and set it as an environment variable for the notebook to use:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}")
```

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
[3]: from google.genai import types

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search, AgentTool, ToolContext
from google.adk.code_executors import BuiltInCodeExecutor

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

1.4: Helper functions

Helper function that prints the generated Python code and results from the code execution tool:

37°C Soleado

ESP LAA 05:53 p. m. 30/04/2026

Day 2a - Agent Tools

```
    "inr": 83.58, # Indian Rupee
  }
}

# Input validation and processing
base = base_currency.lower()
target = target_currency.lower()

# Return structured result with status
rate = rate_database.get(base, {}).get(target)
if rate is not None:
    return {"status": "success", "rate": rate}
else:
    return {
        "status": "error",
        "error_message": f"Unsupported currency pair: {base_currency}/{target_currency}",
    }

print("✅ Exchange rate function created")
print(f"🧪 Test: {get_exchange_rate('USD', 'EUR')}")

✅ Exchange rate function created
🧪 Test: {'status': 'success', 'rate': 0.93}
```

Now let's create our currency agent. Pay attention to how the agent's instructions reference the tools:

37°C Soleado

Day 2a - Agent Tools

```
name="currency_agent",
model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
instruction="""You are a smart currency conversion assistant.

For currency conversion requests:
1. Use 'get_fee_for_payment_method()' to find transaction fees
2. Use 'get_exchange_rate()' to get currency conversion rates
3. Check the 'status' field in each tool's response for errors
4. Calculate the final amount after fees based on the output from 'get_fee_for_payment_method' and '
5. First, state the final converted amount.

Then, explain how you got that result by showing the intermediate amounts. Your explanation must
value in the original currency, the amount remaining after the fee, and the exchange rate used f

If any tool returns status "error", explain the issue to the user clearly.
"""
tools=[get_fee_for_payment_method, get_exchange_rate],
)

print("✅ Currency agent created with custom function tools")
print("\n Available tools:")
print(" • get_fee_for_payment_method - Looks up company fee structure")
print(" • get_exchange_rate - Gets current exchange rates")

✅ Currency agent created with custom function tools
🧪 Available tools:
• get_fee_for_payment_method - Looks up company fee structure
• get_exchange_rate - Gets current exchange rates
```

37°C Soleado

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (4m)

✓ Currency agent created with custom function tools

Available tools:

- get_fee_for_payment_method - Looks up company fee structure
- get_exchange_rate - Gets current exchange rates

```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

### Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?

ValueError                                Traceback (most recent call last)
/tmp/ipykernel_49/3959520931.py in <cell line: 1>()
      1 # Test the currency agent
      2 currency_runner = InMemoryRunner(agent=currency_agent)
----> 3 _ = await currency_runner.run_debug(
      4     "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
      5 )

/usr/local/lib/python3.11/dist-packages/google/adk/runners.py in run_debug(self, user_messages, user_id, session_id, run_config, quiet, verbose)
    1821         print(f'\nuser > {message}')
    1822
    1823
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (4m)

3.1 Built-in Code Executor

ADK has a built-in Code Executor capable of running code in a sandbox. **Note:** This uses Gemini's Code Execution capability.

Let's create a `calculation_agent` which takes in a Python code and uses the `BuiltInCodeExecutor` to run it.

```
calculation_agent = LLMAgent(
    name="CalculationAgent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instructions="""You are a specialized calculator that ONLY responds with Python code. You are forbidden from performing any other actions.

    Your task is to take a request for a calculation and translate it into a single block of Python code that can be executed.

    **RULES:**
    1. Your output MUST be ONLY a Python code block.
    2. Do NOT write any text before or after the code block.
    3. The Python code MUST calculate the result.
    4. The Python code MUST print the final result to stdout.
    5. You are PROHIBITED from performing the calculation yourself. Your only job is to generate the code.

    Failure to follow these rules will result in an error.

    """
)
code_executor=BuiltInCodeExecutor(), # Use the built-in Code Executor Tool. This gives the agent code execution capability
)
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (5m)

```
4. Calculate Final Amount (CRITICAL): You are strictly prohibited from performing any arithmetic calc
code will use the fee information from step 1 and the exchange rate from step 2.
5. Provide Detailed Breakdown: In your summary, you must:
    * State the final converted amount.
    * Explain how the result was calculated, including:
        * The fee percentage and the fee amount in the original currency.
        * The amount remaining after deducting the fee.
        * The exchange rate applied.
    """
    tools=[
        get_fee_for_payment_method,
        get_exchange_rate,
        AgentTool(agent=calculation_agent), # Using another agent as a tool!
    ],
)

print("✅ Enhanced currency agent created")
print("🔗 New capability: Delegates calculations to specialist agent")
print("🔗 Tool types used:")
print("    • Function Tools (fees, rates)")
print("    • Agent Tool (calculation specialist)")
```

Enhanced currency agent created
New capability: Delegates calculations to specialist agent
Tool types used:
• Function Tools (fees, rates)
• Agent Tool (calculation specialist)

+ Code + Markdown

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (5m)

```
print("🔗 Tool types used:")
print("    • Function Tools (fees, rates)")
print("    • Agent Tool (calculation specialist)")

Enhanced currency agent created
New capability: Delegates calculations to specialist agent
Tool types used:
• Function Tools (fees, rates)
• Agent Tool (calculation specialist)
```

```
[33]: # Define a runner
enhanced_runner = InMemoryRunner(agent=enhanced_currency_agent)
```

```
# Test the enhanced agent
response = await enhanced_runner.run_debug(
    "Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation."
)

### Created new session: debug_session_id

User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secr
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 2a - Agent Tools

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (6m)

Congratulations!

You've successfully learned how to build agents that go beyond simple responses to take intelligent actions with custom tools. In this notebook, you learned:

1. **Function Tools** - Converted Python functions into agent tools
2. **Agent Tools** - Created specialist agents and used them as tools
3. **Complete Toolkit** - Explored all ADK tool types and when to use them

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Learn More

Refer to the following documentation to learn more:

- [ADK Documentation](#)
- [ADK Tools Documentation](#)
- [ADK Custom Tools Guide](#)
- [ADK Function Tools](#)
- [ADK Plugins Overview](#)

Next Steps

You've built the foundation of agent tool mastery.

Ready for the next challenge? Continue to the next notebook to learn about **tool patterns**!

Share Save Version 0

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret('GOOGLE_API_KEY')
```

Copy to clipboard

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (0m)

4. Ensure that the checkbox next to 'GOOGLE_API_KEY' is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the 'GOOGLE_API_KEY' you just saved and set it as an environment variable for the notebook to use:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print("Setup and authentication complete.")
except Exception as e:
    print(f"Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets.")
```

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit. This keeps your code organized and ensures we have

Share Save Version 0

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret('GOOGLE_API_KEY')
```

Copy to clipboard

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

The screenshot shows the Kaggle AI interface with a code cell titled "# MCP integration with Everything Server". The code is as follows:

```
# MCP integration with Everything Server
Execute cell. Cell execution started at 5:35pm.
set(
  connection_params=StdioConnectionParams(
    server_params=StdioServerParameters(
      command="npx", # Run MCP server via npx
      args=[
        "-y", # Argument for npx to auto-confirm install
        "@modelcontextprotocol/server-everything",
      ],
      tool_filter=["getTinyImage"],
    ),
    timeout=30,
  )
)

print("✅ MCP Tool created")
```

The output of the code execution is:

```
✅ MCP Tool created
```

The interface includes a sidebar with "Add Secrets" and a "Save Version" button. The bottom status bar shows the temperature is 29°C and the location is Maymorn, nublado.

[illegible]

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (7m)

When you resume, the App loads this saved state so the agent continues exactly where it left off - as if no time passed.

```
# Wrap the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
name="shipping_coordinator",
root_agent=shipping_agent,
resumability_config=ResumabilityConfig(is_resumable=True),

print("✅ Resumable app created!")

Resumable app created!
/tmp/ipykernel_49/36737775.py:5: UserWarning: [EXPERIMENTAL] ResumabilityConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  resumability_config=ResumabilityConfig(is_resumable=True),
```

+ Code + Markdown

Step 3: Create Session and Runner with the App

Pass `app=shipping_app` instead of `agent=...` so the runner knows about resumability.

```
[ ]: session_service = InMemorySessionService()
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm, nublado

Day 2b - Agent Tools Best Practices

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (7m)

`create_approval_response()` - Formats the human decision

- Takes the approval info and boolean decision (True/False) from the human
- Creates a `FunctionResponse` that ADK understands
- Wraps it in a `Content` object to send back to the agent

```
def create_approval_response(approval_info, approved):
    """Create approval response message."""
    confirmation_response = types.FunctionResponse(
        id=approval_info["approval_id"],
        name="adk_request_confirmation",
        response={"confirmed": approved},
    )
    return types.Content(
        role="user", parts=[types.Part(function_response=confirmation_response)]
    )

print("✅ Helper functions defined")

Helper functions defined
```

+ Code + Markdown

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm, nublado

Day 3a - Agent Sessions Draft saved

File Edit View Run Settings Add-ons Help

Cancel Run Code Draft Session (11m)

3. Authenticate in the notebook

Run the cell below to complete authentication.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
    )
```

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

An error occurred updating the session.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (12m)

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from typing import Any, Dict

from google.adk.agents import Agent, LlmAgent
from google.adk.apps.app import App, EventsCompactionConfig
from google.adk.models.google_llm import Gemini
from google.adk.sessions import DatabaseSessionService
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.adk.tools.tool_context import ToolContext
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

1.4: Helper functions

Helper function that manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming. It supports both single queries and multiple queries in sequence.

Example:

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3a - Agent Sessions Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (12m)

```
# Process each query in the list sequentially
for query in user_queries:
    print(f"\nUser > {query}")

    # Convert the query string to the ADK Content format
    query = types.Content(role="user", parts=[types.Part(text=query)])

    # Stream the agent's response asynchronously
    async for event in runner_instance.run_async(
        user_id=USER_ID, session_id=session_id, new_message=query
    ):
        # Check if the event contains valid content
        if event.content and event.content.parts:
            # Filter out empty or "None" responses before printing
            if (
                event.content.parts[0].text != "None"
                and event.content.parts[0].text
            ):
                print(f"(MODEL_NAME) > ", event.content.parts[0].text)
            else:
                print("No queries!")

    print("✅ Helper functions defined.")

Helper functions defined.
```

+ Code + Markdown

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

Ulovima Por la noche

Búsqueda

05:54 p. m. 02/05/2026

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Markdown Draft Session (13m)

handle these failures by retrying the request with exponential backoff.

```
[7]: retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

```
[8]: retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

Section 2: Session Management

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

Puesta de sol 18:45

Búsqueda

05:54 p. m. 02/05/2026

Day 3a - Agent Sessions

```
MODEL_NAME = "gemini-2.5-flash-lite"

# Step 1: Create the LLM Agent
root_agent = Agent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot", # Description of the agent's purpose
)

# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary)
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Stateful agent initialized!")
print(f"  - Application: {APP_NAME}")
print(f"  - User: {USER_ID}")
print(f"  - Using: {session_service.__class__.__name__}")

✅ Stateful agent initialized!
  - Application: default
  - User: default
  - Using: InMemorySessionService
```

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 3a - Agent Sessions

2.5 testing Our Stateful Agent

Now let's see the magic of sessions in action!

```
# Run a conversation with two queries in the same session
await run_session(
    runner,
    [
        "Hi, I am Sam! What is the capital of United States?",
        "Hello! What is my name?", # This time, the agent should remember!
    ],
    "stateful-agentic-session",
)

### Session: stateful-agentic-session

User > Hi, I am Sam! What is the capital of United States?

ValueError                                Traceback (most recent call last)
/tmp/ipykernel_49/1004658795.py in <cell line: 1>()
      1 # Run a conversation with two queries in the same session
      2 # Notice: Both queries are part of the SAME session, so context is maintained
----> 3 await run_session(
      4     runner,
      5     [
```

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (13m)

Success! The agent remembered your name because both queries were part of the same session. The Runner automatically maintained the conversation history.

But there's a catch: `InMemorySessionService` is temporary. Once the application stops, all conversation history is lost.

(Optional) 2.6 Testing Agent's forgetfulness

To verify that the agent forgets the conversation, restart the kernel. Then, run ALL the previous cells in this notebook EXCEPT the `run_session` in 2.5.

Now run the cell below. You'll see that the agent doesn't remember anything from the previous conversation.

```
# Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    ["What did I ask you about earlier?", "And remind me, what's my name?"],
    "stateful-agentic-session",
) # Note, we are using same session name

### Session: stateful-agentic-session
User > What did I ask you about earlier?
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Puesta de sol 18:45

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (14m)

Let's upgrade to `DatabaseSessionService` using SQLite. This gives us persistence without needing a separate database server for this demo.

Let's create a `chatbot_agent` capable of having a conversation with the user.

```
# Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f"  - Database: my_agent_data.db")
print(f"  - Sessions will survive restarts!")

✅ Upgraded to persistent sessions!
  - Database: my_agent_data.db
  - Sessions will survive restarts!
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

29°C Mayorm. nublado

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

+

Run All

Code

Draft Session (14m)

```
#!/usr/local/lib/python3.11/dist-packages/google/genai/_api_client.py in __init__(self, vertexai, api_key, credentials, project, location, http_options)
674     else: # Implicit initialization or missing arguments.
675         if not self.api_key:
676             raise ValueError(
677                 "Missing key inputs argument! To use the Google AI API, '
678                 provide ('api_key') arguments. To use the Google Cloud API, '
ValueError: Missing key inputs argument! To use the Google AI API, provide ('api_key') arguments. To use the Google Cloud API, provide ('vertexai', 'project' & 'location') arguments.
```

+ Code + Markdown

3.5 Let's verify that the session data is isolated

As mentioned earlier, a session is private conversation between an Agent and a User (i.e., two sessions do not share information). Let's run our `run_session` with a different session name `'test-db-session-02'` to confirm this.

```
await run_session(
    runner, ["Hello! What is my name?"], "test-db-session-02"
) # Note, we are using new session name
```

Session: test-db-session-02

User > Hello! What is my name?

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient()
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

.....

☒ GOOGLE_API_KEY

.....

+ Add Secret

Copy snippet

29°C Mayorm, nublado

Búsqueda

05:56 p. m. 02/05/2026

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

+

Run All

Code

Draft Session (15m)

```
# Re-define our app with Events Compaction enabled
research_app_compacting = App(
    name="research_app_compacting",
    root_agent=chatbot_agent,
    # This is the new part!
    events_compaction_config=EventsCompactionConfig(
        compaction_interval=3, # Trigger compaction every 3 invocations
        overlap_size=1, # Keep 1 previous turn for context
    ),
)

db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Create a new runner for our upgraded app
research_runner_compacting = Runner(
    app=research_app_compacting, session_service=session_service
)

print("✅ Research App upgraded with Events Compaction!")
```

Research App upgraded with Events Compaction!

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient()
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

.....

☒ GOOGLE_API_KEY

.....

+ Add Secret

Copy snippet

29°C Mayorm, nublado

Búsqueda

05:56 p. m. 02/05/2026

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (15m)

```
# Get the final session state
Execute cell. Cell executed in 0.023s at 5:57pm
app_name=ResearchRunner_compacting.app_name,
user_id=USER_ID,
session_id="compaction_demo",
)

print("--- Searching for Compaction Summary Event ---")
found_summary = False
for event in final_session.events:
    # Compaction events have a 'compaction' attribute
    if event.actions and event.actions.compaction:
        print("\n✅ SUCCESS! Found the Compaction Event:")
        print(f"  Author: {event.author}")
        print(f"  Info: {event.info}")
        found_summary = True
        break

if not found_summary:
    print("\n❌ No compaction event found. Try increasing the number of turns in the demo.")

--- Searching for Compaction Summary Event ---
AttributeError: Traceback (most recent call last)
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm, nublado

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (16m)

```
Args:
  user_name: The username to store in session state
  country: The name of the user's country
"""
# Write to session state using the 'user:' prefix for user data
tool_context.state["user:name"] = user_name
tool_context.state["user:country"] = country

return {"status": "success"}

# This demonstrates how tools can read from session state.
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to retrieve user name and country from session state.
    """
    # Read from session state
    user_name = tool_context.state.get("user:name", "Username not found")
    country = tool_context.state.get("user:country", "Country not found")

    return {"status": "success", "user_name": user_name, "country": country}

print("✅ Tools created.")

Tools created.
```

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm, nublado

Day 3a - Agent Sessions Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (16m)

```
# Configuration
APP_NAME = "default"
USER_ID = "default"
MODEL_NAME = "gemini-2.5-flash-lite"

# Create an agent with session state tools
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot.
    Tools for managing user context:
    * To record username and country when provided use 'save_userinfo' tool.
    * To fetch username and country when required use 'retrieve_userinfo' tool.
    ...
    tools=[save_userinfo, retrieve_userinfo], # Provide the tools to the agent
)

# Set up session service and runner
session_service = InMemorySessionService()
runner = Runner(agent=root_agent, session_service=session_service, app_name="default")

print("Agent with session state tools initialized!")
```

Agent with session state tools initialized!

+ Code + Markdown

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm. nublado

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (16m)

```
["Hi there, how are you doing today? What is my name?"].
"new-isolated-session",

# Expected: The agent won't know the name because this is a different session
```

5.6 Cross-Session State Sharing

While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session (new-isolated-session):

```
# Check the state of the new session
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)

print("New Session State:")
print(session.state)

# Note: Depending on implementation, you might see shared state here.
# This is where the distinction between session-specific and user-specific state becomes important.
```

New Session State:

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

29°C Mayorm. nublado

Day 3a - Agent Sessions

File Edit View Run Settings Add-ons Help

Run All Code

Production Considerations - You're ready to handle real-world challenges

Congratulations! You did it

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Learn More

Refer to the following documentation to learn more:

- ADK Documentation
- ADK Sessions
- ADK Session-State
- ADK Session Compaction

Next Steps - Long Term Memory Systems (Part 2)

Why do we need memory?

In this notebook, we manually identified a couple characteristic (username and country) and built tools to manage it. But real conversations involve hundreds of such characteristics:

- Use performance and habits

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret('GOOGLE_API_KEY')
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 3b - Agent Memory

File Edit View Run Settings Add-ons Help

Run All Code

4. ensure that the checkbox next to 'GOOGLE_API_KEY' is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to complete authentication.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret('GOOGLE_API_KEY')
    os.environ['GOOGLE_API_KEY'] = GOOGLE_API_KEY
    print("Gemini API key setup complete.")
except Exception as e:
    print(f"Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}")
```

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

Day 3b - Agent Memory Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (1m)

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.genai import types

print("ADK components imported successfully.")
```

ADK components imported successfully.

1.4: Helper functions

This helper function manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming.

29°C Mayorm, nublado

Day 3b - Agent Memory Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

Draft Session (1m)

Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: Connection error trying to communicate with service.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.genai import types

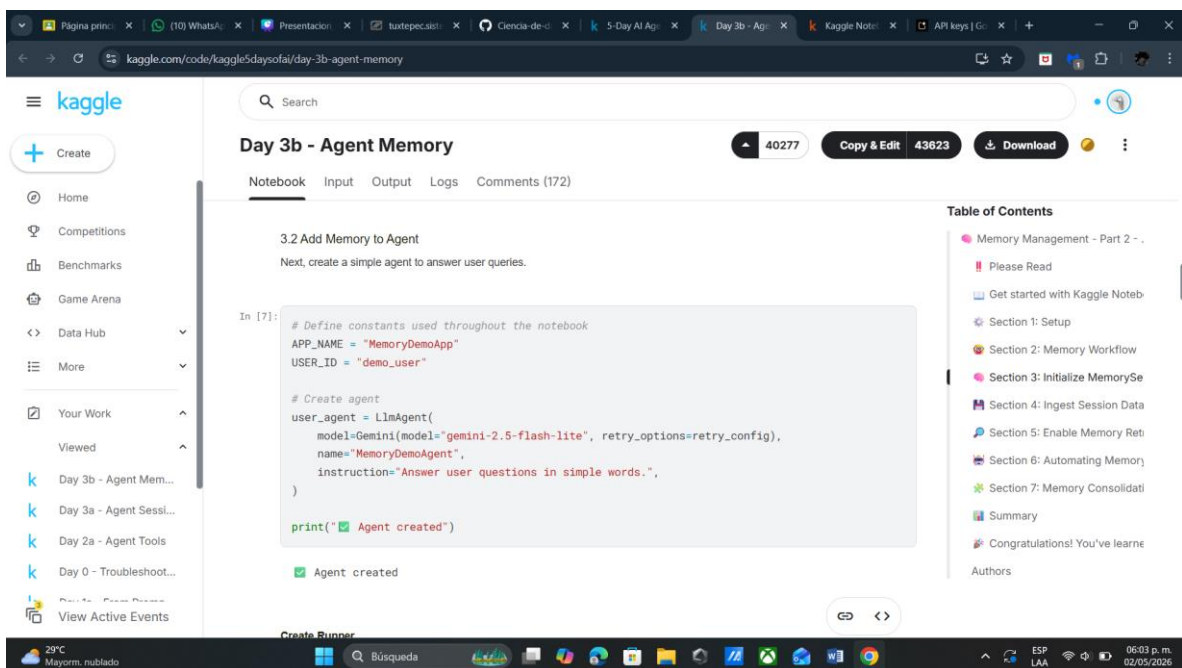
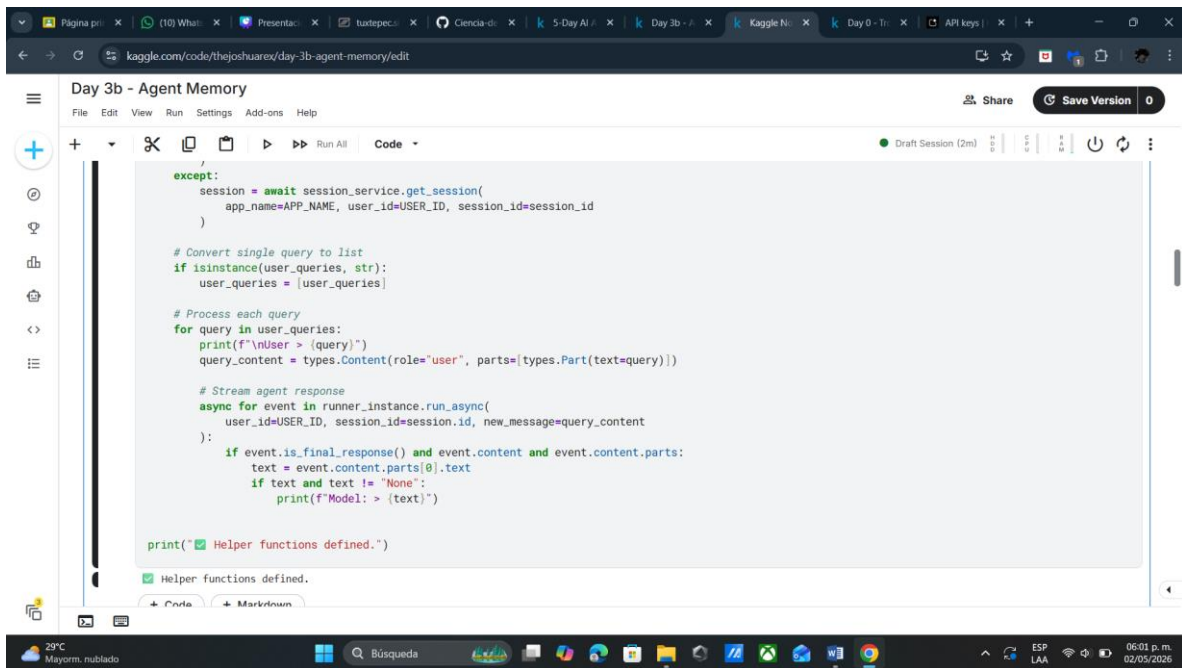
print("ADK components imported successfully.")
```

ADK components imported successfully.

1.4: Helper functions

This helper function manages a complete conversation session, handling session creation/retrieval, query processing, and response streaming.

29°C Mayorm, nublado



Browser tabs: Página princ..., (10) WhatsA..., Presentación..., Iuxtepec:ci..., Ciencia de..., 5-Day AI Ag..., Day 3b - Ag..., Kaggle Note..., API keys | G..., +

URL: kaggle.com/code/kaggle5daysofai/day-3b-agent-memory

Kaggle logo

+ Create

Home

Competitions

Benchmarks

Game Arena

Data Hub

More

Your Work

Viewed

Day 3b - Agent Mem...

Day 3a - Agent Sessi...

Day 2a - Agent Tools

Day 0 - Troubleshoot...

View Active Events

Search

Day 3b - Agent Memory

40277 Copy & Edit 43623 Download

Notebook Input Output Logs Comments (172)

```
In [8]: # Create Session Service
session_service = InMemorySessionService() # Handles conversations

# Create runner with BOTH services
runner = Runner(
    agent=user_agent,
    app_name="MemoryDemoApp",
    session_service=session_service,
    memory_service=memory_service, # Memory service is now available!
)

print("✅ Agent and Runner created with memory support!")

✅ Agent and Runner created with memory support!
```

Important

Configuration vs. Usage: Adding `memory_service` to the `Runner` makes memory available to your agent, but doesn't automatically use it. You must explicitly:

Table of Contents

- Memory Management - Part 2 -
- Please Read
- Get started with Kaggle Noteb
- Section 1: Setup
- Section 2: Memory Workflow
- Section 3: Initialize MemorySe
- Section 4: Ingest Session Data
- Section 5: Enable Memory Reti
- Section 6: Automating Memory
- Section 7: Memory Consolida
- Summary
- Congratulations! You've learne
- Authors

29°C Mayorm, nublado

Browser tabs: Página princ..., (10) WhatsA..., Presentación..., Iuxtepec:ci..., Ciencia de..., 5-Day AI Ag..., Day 3b - Ag..., Kaggle Note..., API keys | G..., +

URL: kaggle.com/code/kaggle5daysofai/day-3b-agent-memory

Kaggle logo

+ Create

Home

Competitions

Benchmarks

Game Arena

Data Hub

More

Your Work

Viewed

Day 3b - Agent Mem...

Day 3a - Agent Sessi...

Day 2a - Agent Tools

Day 0 - Troubleshoot...

View Active Events

Search

Day 3b - Agent Memory

40277 Copy & Edit 43623 Download

Notebook Input Output Logs Comments (172)

```
In [10]: session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="conversation-01"
)

# Let's see what's in the session
print("📄 Session contains:")
for event in session.events:
    text = (
        event.content.parts[0].text[:60]
        if event.content and event.content.parts
        else "(empty)"
    )
    print(f"📄 {event.content.role}: {text}...")

📄 Session contains:
user: My favorite color is blue-green. Can you write a Haiku about...
model: A tranquil blend,
Ocean's calm and nature's green,
Peace in ...
```

Table of Contents

- Memory Management - Part 2 -
- Please Read
- Get started with Kaggle Noteb
- Section 1: Setup
- Section 2: Memory Workflow
- Section 3: Initialize MemorySe
- Section 4: Ingest Session Data
- Section 5: Enable Memory Reti
- Section 6: Automating Memory
- Section 7: Memory Consolida
- Summary
- Congratulations! You've learne
- Authors

29°C Mayorm, nublado

5.2 Add Load Memory Tool to Agent

Let's start by implementing the reactive pattern. We'll recreate the agent from Section 3, this time adding the `load_memory` tool to its toolkit. Since this is a built-in ADK tool, you simply include it in the tools array without any custom implementation.

```
In [12]: # Create agent
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words. Use load_memory tool if you need to rec
all past conversations.",
    tools=[
        load_memory
    ], # Agent now has access to Memory and can search it whenever it decides to!
)

print("✅ Agent with load_memory tool created.")
```

✅ Agent with load_memory tool created.

Table of Contents

- Memory Management - Part 2 - .
- Please Read
- Get started with Kaggle Noteb
- Section 1: Setup
- Section 2: Memory Workflow
- Section 3: Initialize MemorySe
- Section 4: Ingest Session Data
- Section 5: Enable Memory Reti
- Section 6: Automating Memory
- Section 7: Memory Consolidati
- Summary
- Congratulations! You've learne
- Authors

5.3 Update the Runner and Test

The screenshot displays a Kaggle Notebook environment. The top bar shows the system clock at 06:04 p.m. on 02/05/2026. The notebook interface includes a left sidebar with navigation options like Home, Competitions, Benchmarks, Game Arena, Data Hub, and More. The main area is titled 'Notebook' and shows the execution of a code cell (In [17]). The code searches for color preferences using the `memory_service.search_memory` function. The output shows 'Search Results: Found 4 relevant memories' and a conversation snippet where the user asks for a Haiku about their favorite color (blue-green) and the agent responds with a poem.

```
In [17]: # Search for color preferences
search_response = await memory_service.search_memory(
    app_name=APP_NAME, user_id=USER_ID, query="What is the user's favorite color?"
)

print("🔍 Search Results:")
print(f" Found {len(search_response.memories)} relevant memories")
print()

for memory in search_response.memories:
    if memory.content and memory.content.parts:
        text = memory.content.parts[0].text[:80]
        print(f" [{memory.author}]: {text}...")
```

🔍 Search Results:
Found 4 relevant memories

[user]: My favorite color is blue-green. Can you write a Haiku about it?...

[MemoryDemoAgent]: A tranquil blend,
Ocean's calm and nature's green,
Peace in every hue....

Day 3b - Agent Memory 35eac8 Draft saved

File Edit View Run Settings Add-ons Help

+ Draft Session off (run a cell to start)

Congratulations! You've learned Memory Management in ADK!

Learn More:

- ADK Memory Documentation
- Vertex AI Memory Bank
- Memory Consolidation Guide

Next Steps:

Ready for Day 4? Learn how to **implement Observability and Evaluate your agents** to ensure they're working as intended in production!

Authors

Sampath M

Notebook

Input

+ Add Input Upload

No input attached

Attach a Kaggle dataset, model, or competition

Output

/kaggle/working

Table of contents

29°C Mayorm, nublado

Day 4a - Agent Observabil... Draft saved

File Edit View Run Settings Add-ons Help

+ Draft Session (0m)

1.3: Set up logging and cleanup old files

Let's configure logging for our debugging session. The following cell makes sure we also capture other log levels, like DEBUG.

```
import logging
import os

# Clean up any previous logs
for log_file in ["logger.log", "web.log", "tunnel.log"]:
    if os.path.exists(log_file):
        os.remove(log_file)
        print(f"Cleaned up {log_file}")

# Configure logging with DEBUG log level.
logging.basicConfig(
    filename="logger.log",
    level=logging.DEBUG,
    format="%(filename)s:%(lineno)s %(levelname)s: %(message)s",
)

print("Logging configured")
```

Logging configured

29°C Mayorm, nublado

Day 4a - Agent Observabil...

```
<div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
  The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
  <ol style="margin-top: 10px; padding-left: 20px;">
    <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!adk web ...</code>) to start the ADK web UI.</li>
    <li style="margin-bottom: 5px;"><strong>Wait for that cell to show it is "Running" (it will not "complete").</li>
    <li>Once it's running, <strong>return to this button</strong> and click it to open the UI.</li>
  </ol>
  <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
</div>
<a href="{url}" target="_blank" style="
  display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
  text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
  box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
  Open ADK Web UI (after running cell below) </a>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")
```

Helper functions defined.

Day 4a - Agent Observabil... Draft saved

Note: The following cell will not "complete", but will remain running and serving the ADK web UI until you manually stop the cell.

```
!adk web --log_level DEBUG --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCred
entialService: This feature is experimental and may change or be removed in future versions without notice. It m
ay introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: U
serWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in f
uture versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [84]
INFO: Waiting for application startup.

-----
| ADK Web Server started
| For local testing, access at http://127.0.0.1:8000.
|-----

INFO: Application startup complete.
```

Once the ADK web UI starts, open the proxy link using the button in the previous cell.

Armoury Crate

El nivel de la batería es inferior al 50%. Vaya a Armoury Crate si desea detener las aplicaciones que actualmente estén utilizando la dGPU.

Aceptar Más información

Day 4a - Agent Observabil... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (0m)

```
google_search_agent = LLMAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="Searches for information using Google search",
    instruction="Use the google_search tool to find information on the given topic. Return the raw search results",
    tools=[google_search],
)

# Root agent
research_agent_with_plugin = LLMAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="Your task is to find research papers and count them.",
    tools=[AgentTool(agent=google_search_agent, count_papers)],
)

print("Agent created")
```

Agent created

+ Code + Markdown

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

26°C Nublado

Day 4a - Agent Observabil... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (1m)

To use `LoggingPlugin` in the above research agent,

1. Import the plugin
2. Add it when initializing the `InMemoryRunner`.

```
from google.adk.runners import InMemoryRunner
from google.adk.plugins.logging_plugin import (
    LoggingPlugin,
) # <---- 1. Import the Plugin
from google.genai import types
import asyncio

runner = InMemoryRunner(
    agent=research_agent_with_plugin,
    plugins=[
        LoggingPlugin()
    ], # <---- 2. Add the plugin. Handles standard Observability logging across ALL agents
)

print("Runner configured")
```

Runner configured

+ Code + Markdown

Let's now run the agent using `run_debug` function.

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

26°C Nublado

Day 4b - Agent Evaluation

```
print(" RESPONSE QUALITY: Poor - Response text too different from expected")
print(" ROOT CAUSE: Agent's communication style, not functionality")
print()
print(" ACTIONABLE INSIGHTS:")
print("1. Technical capability works (tool usage perfect)")
print("2. Communication needs improvement (response quality failed)")
print("3. Fix: Update agent instructions for clearer language or constrained response.")
print()
```

Understanding Evaluation Results:

EXAMPLE ANALYSIS:

Test Case: living_room_light_on

- response_match_score: 0.45/0.80
- tool_trajectory_avg_score: 1.0/1.0

What this tells us:

- TOOL USAGE: Perfect - Agent used correct tool with correct parameters
- RESPONSE QUALITY: Poor - Response text too different from expected
- ROOT CAUSE: Agent's communication style, not functionality

ACTIONABLE INSIGHTS:

1. Technical capability works (tool usage perfect)
2. Communication needs improvement (response quality failed)
3. Fix: Update agent instructions for clearer language or constrained response.

+ Code + Markdown

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 4b - Agent Evaluation

Congratulations!

You've learned

- Interactive test creation and analysis in the ADK web UI
- Tool trajectory and response metrics
- Automated regression testing using `adk eval` CLI command
- How to analyze evaluation results and fix agents based on it

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Resources

- ADK Evaluation overview
- Different evaluation criteria
- Pytest based Evaluation

Advanced Evaluation

For production deployments, ADK supports advanced criteria like `safety_v1` and `hallucinations_v1` (requires Google Cloud credentials).

Next Steps

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 5a - Agent2Agent Communicati... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (5m)

3. Paste your API key into the "Value" field and click "Save".

4. Ensure that the checkbox next to `GOOGLE_API_KEY` is selected so that the secret is attached to the notebook.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets")
```

✅ Setup and authentication complete.

+ Code + Markdown

Import ADK components

New: Import the specific components you'll need from the Agent Development Kit and the Generative AI library. This leaves some code.

Add: Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

26°C Nublado Búsqueda

Day 5a - Agent2Agent Communicati... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (8m)

Execute cell. Cell executed in 73.50% at 4:27pm

```
import json
import subprocess
import time
import uuid

from google.adk.agents import LlmAgent
from google.adk.agents.remote_a2a_agent import (
    RemoteA2aAgent,
    AGENT_CARD_WELL_KNOWN_PATH,
)

from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

# Hide additional warnings in the notebook
import warnings
warnings.filterwarnings("ignore")

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

Add: Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

26°C Nublado Búsqueda

Day 5a - Agent2Agent Communication

```
# Create the Product Catalog Agent
# This agent specializes in providing product information from the vendor's catalog
product_catalog_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="product_catalog_agent",
    description="External vendor's product catalog agent that provides product information and availability",
    instruction="""
    You are a product catalog specialist from an external vendor.
    When asked about products, use the get_product_info tool to fetch data from the catalog.
    Provide clear, accurate product information including price, availability, and specs.
    If asked about multiple products, look up each one.
    Be professional and helpful.
    """
    tools=[get_product_info], # Register the product lookup tool
)

print("✅ Product Catalog Agent created successfully!")
print("Model: gemini-2.5-flash-lite")
print("Tool: get_product_info()")
print("Ready to be exposed via A2A...")
```

✅ Product Catalog Agent created successfully!
Model: gemini-2.5-flash-lite
Tool: get_product_info()
Ready to be exposed via A2A...

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 5a - Agent2Agent Communication

A2A Protocol Specification

```
# Convert the product catalog agent to an A2A-compatible application
# This creates a FastAPI/Starlette app that:
# 1. Serves the agent at the A2A protocol endpoints
# 2. Provides an auto-generated agent card
# 3. Handles A2A communication protocol
product_catalog_a2a_app = to_a2a(
    product_catalog_agent, port=8001 # Port where this agent will be served
)

print("✅ Product Catalog Agent is now A2A-compatible!")
print("Agent will be served at: http://localhost:8001")
print("Agent card will be at: http://localhost:8001/.well-known/agent-card.json")
print("Ready to start the server...")
```

✅ Product Catalog Agent is now A2A-compatible!
Agent will be served at: http://localhost:8001
Agent card will be at: http://localhost:8001/.well-known/agent-card.json
Ready to start the server...

Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

+ Add Secret Copy snippet

Day 5a - Agent2Agent Communication

```
print("Waiting for server to be ready...")

# Wait for server to start (poll until it responds)
max_attempts = 30
for attempt in range(max_attempts):
    try:
        response = requests.get(
            "http://localhost:8001/.well-known/agent-card.json", timeout=1
        )
        if response.status_code == 200:
            print(f"Product Catalog Agent server is running!")
            print(f"Server URL: http://localhost:8001")
            print(f"Agent card: http://localhost:8001/.well-known/agent-card.json")
            break
    except requests.exceptions.RequestException:
        time.sleep(5)
        print(".", end="", flush=True)
    else:
        print("\n⚠️ Server may not be ready yet. Check manually if needed.")

# Store the process so we can stop it later
globals()["product_catalog_server_process"] = server_process
```

Product Catalog agent code saved to /tmp/product_catalog_server.py
Starting Product Catalog Agent server...
Waiting for server to be ready...

Product Catalog Agent server is running!
Server URL: http://localhost:8001
Agent card: http://localhost:8001/.well-known/agent-card.json

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 5a - Agent2Agent Communication

```
response = requests.get(
    "http://localhost:8001/.well-known/agent-card.json", timeout=1
)
if response.status_code == 200:
    print(f"Product Catalog Agent server is running!")
    print(f"Server URL: http://localhost:8001")
    print(f"Agent card: http://localhost:8001/.well-known/agent-card.json")
    break
except requests.exceptions.RequestException:
    time.sleep(5)
    print(".", end="", flush=True)
else:
    print("\n⚠️ Server may not be ready yet. Check manually if needed.")

# Store the process so we can stop it later
globals()["product_catalog_server_process"] = server_process
```

Product Catalog agent code saved to /tmp/product_catalog_server.py
Starting Product Catalog Agent server...
Waiting for server to be ready...
Product Catalog Agent server is running!
Server URL: http://localhost:8001
Agent card: http://localhost:8001/.well-known/agent-card.json

View the Auto-Generated Agent Card

This function automatically created an agent card that describes the Product Catalog Agent's capabilities. Let's take a look.

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 5a - Agent2Agent Communication

```
print(f" Name: {agent_card.get('name')}")
print(f" Description: {agent_card.get('description')}")
print(f" URL: {agent_card.get('url')}")
print(f" Skills: {len(agent_card.get('skills', []))} capabilities exposed")
else:
    print(f"❌ Failed to fetch agent card: {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"❌ Error fetching agent card: {e}")
    print(" Make sure the Product Catalog Agent server is running (previous cell)")

Product Catalog Agent Card:
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "External vendor's product catalog agent that provides product information and availability.",
  "name": "product_catalog_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
    {
      "description": "External vendor's product catalog agent that provides product information and availability.
      \n I am a product catalog specialist from an external vendor.\n When asked about products, use the get_produ
      ct_info tool to fetch data from the catalog.\n Provide clear, accurate product information including price, ava
      ilability, and specs.\n If asked about multiple products, look up each one.\n Be professional and helpful.\n
    }
  ]
}
```

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 5a - Agent2Agent Communication

```
# Create a RemoteA2Agent that connects to our Product Catalog Agent
# This acts as a client-side proxy - the Customer Support Agent can use it like a local agent
remote_product_catalog_agent = RemoteA2Agent(
    name="product_catalog_agent",
    description="Remote product catalog agent from external vendor that provides product information.",
    # Point to the agent card URL - this is where the A2A protocol metadata lives
    agent_card=f"http://localhost:8001/{AGENT_CARD_WELL_KNOWN_PATH}",
)

print("✅ Remote Product Catalog Agent proxy created!")
print(f" Connected to: http://localhost:8001")
print(f" Agent card: http://localhost:8001/{AGENT_CARD_WELL_KNOWN_PATH}")
print(" The Customer Support Agent can now use this like a local sub-agent!")

Remote Product Catalog Agent proxy created!
Connected to: http://localhost:8001
Agent card: http://localhost:8001/well-known/agent-card.json
The Customer Support Agent can now use this like a local sub-agent!

# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LlmAgent(
    model=Gemini(model="gemini-1.5-flash-lite", retry_options=retry_config))
```

Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

Gemini API Key

GOOGLE_API_KEY

Add Secret Copy snippet

Day 5a - Agent2Agent Communication... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (16m)

```
# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="customer_support_agent",
    description="A customer support assistant that helps customers with product inquiries and information",
    instruction="""
    You are a friendly and professional customer support agent.

    When customers ask about products:
    1. Use the product_catalog_agent sub-agent to look up product information
    2. Provide clear answers about pricing, availability, and specifications
    3. If a product is out of stock, mention the expected availability
    4. Be helpful and professional!

    Always get product information from the product_catalog_agent before answering customer questions.
    """,
    sub_agents=[remote_product_catalog_agent], # Add the remote agent as a sub-agent!
)

print("✅ Customer Support Agent created!")
print("Model: gemini-2.5-flash-lite")
print("Sub-agents: 1 (remote Product Catalog Agent via A2A)")
print("Ready to help customers!")
```

✅ Customer Support Agent created!
Model: gemini-2.5-flash-lite
Sub-agents: 1 (remote Product Catalog Agent via A2A)

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

26°C Nublado

Day 5a - Agent2Agent Communication... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code Draft Session (18m)

```
# Create the user message
# This follows the same pattern as the deployment notebook
test_content = types.Content(parts=[types.Part(text=user_query)])

# Display query
print(f"\n👤 Customer: {user_query}")
print(f"\n🤖 Support Agent response:")
print("-" * 60)

# Run the agent asynchronously (handles streaming responses and A2A communication)
async for event in runner.run_async(
    user_id=user_id, session_id=session_id, new_message=test_content
):
    # Print final response only (skip intermediate events)
    if event.is_final_response() and event.content:
        for part in event.content.parts:
            if hasattr(part, "text"):
                print(part.text)

print("-" * 60)

# Run the test
print("🧪 Testing A2A Communication...\n")
await test_a2a_communication("Can you tell me about the iPhone 15 Pro? Is it in stock?")
```

🧪 Testing A2A Communication...

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key

☒ GOOGLE_API_KEY

+ Add Secret Copy snippet

26°C Nublado

Page: x

(10) x

Pres: x

task: x

Clm: x

5-D: x

k Day: x

k Kag: x

local: x

k Kag: x

Jupy: x

Find: x

API: x

+ x

kaggle.com/code/thejoshuaex/day-5a-agent2agent-communication/edit

Share Save Version 0

Day 5a - Agent2Agent Communicati...

File Edit View Run Settings Add-ons Help

+ - ✂ 📄 ▶ ▶▶ Run All Code

Draft Session (19m)

⏻ ↺ ⋮

+

🔍

🔗

📊

📄

<>

☰

🟢 Congratulations! You're an A2A Expert

You've successfully learned how to build multi-agent systems using the A2A protocol!

You now know how to expose agents as services, consume remote agents, and build collaborative multi-agent systems that can scale across teams and organizations.

Note: No submission required!

This notebook is for your hands-on practice and learning only. You **do not** need to submit it anywhere to complete the course.

Learn More

Refer to the following documentation to learn more:

- ADK Documentation
- A2A Protocol Official Website
- A2A Tutorials
- Introduction to A2A in ADK
- Exposing Agents Quickstart
- Consuming Agents Quickstart

Next Steps

Now that you understand A2A communication, you can build complex multi-agent systems where specialized agents collaborate to solve

Add Secrets

Code Snippet

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copy to clipboard

☐ Gemini API Key *****

☒ GOOGLE_API_KEY *****

+ Add Secret Copy snippet

26°C Nublado

Busqueda

ESP LAA 04:38 p. m. 03/05/2026